



PlugSure CSMS

Deployment Guide

Version 1.3.0

Enterprise Operator Console — installation, configuration, first charger, upgrade and operations

Package	<code>plugsure-v1.3.0.zip</code>
Implements	SPEC-UI-CSMS-2026-FINAL (Modules 1–10)
Stack	Node.js 22 · PostgreSQL 16 · Caddy 2 · OCPP 1.6-J, 2.0.1 & 2.1
Verified	Re-run on PostgreSQL 16 on 29 September 2026, after the running-cost and security fixes: typecheck clean · 571/571 unit tests · 96/96 console + 50/50 driver-app + 147/147 field-conditions + 17/17 Live Activities + 22/22 integrations + 21/21 e-wallet & card payments + 38/38 card holds & saved cards + 19/19 linked e-wallets + 40/40 post-pay + 24/24 promotions & memberships + 21/21 bidirectional charging end-to-end checks. From the 27–28 September runs (PostgreSQL 18), not re-run since: 46/46 map, push & reservations + 19/19 driver queue + 12/12 reservation fees + 74/74, 60/60 and 36/36 roaming + 38/38 API & sandbox + 23/23 SDK & rate limits + 20/20 signed meter values + 17/17 sandbox OCPP 2.x + 28/28 white-label driver apps + 20/20 iOS notifications + 53/53 fleet billing + 37/37 Plug & Charge + 19/19 onboarding.
Date	29 September 2026
Audience	Deployment engineers and the platform operations team

Contents

1. Overview

2. Architecture

3. Prerequisites

4. Prepare the configuration

5. Install — Path A: Docker Compose

6. Install — Path B: systemd (no Docker)

7. Caddy, DNS and firewall

8. Create the first administrator
9. Verify the installation

10. Commission the first charger

11. Upgrading from v1.2.1

12. Operations

13. Troubleshooting

14. Go-live checklist

15. Configuration reference

1. Overview

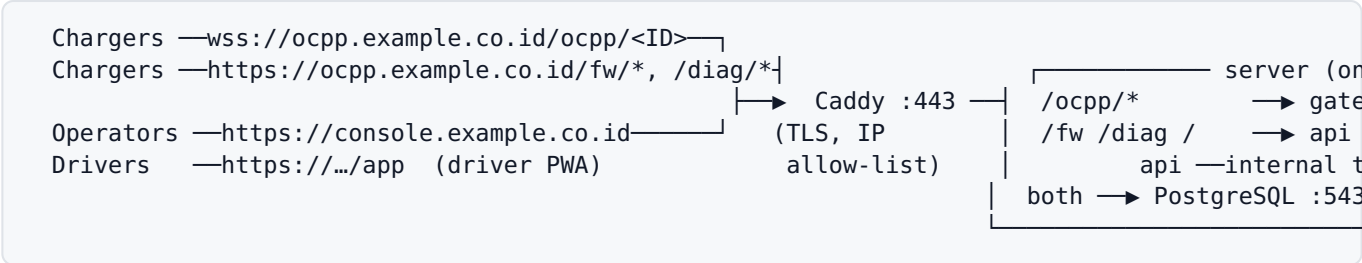
PlugSure is deployed as two Node.js processes built from the same package, plus PostgreSQL and a Caddy TLS edge:

Component	Port	Role
OCPP gateway (<code>dist/apps/gateway.js</code>)	9220	Charger WebSockets (OCPP 1.6-J / 2.0.1). Also runs the background workers: load-management control loop, compliance sweep, session reconciliation, firmware (FOTA) scheduler.
API + operator console (<code>dist/apps/api.js</code>)	9200	REST API, the web console at <code>/</code> , the driver app at <code>/app</code> , and the charger-facing firmware download (<code>/fw/*</code>) and log upload (<code>/diag/*</code>) endpoints.
PostgreSQL 16	5432	All data. Loopback only.
Caddy 2	443	TLS for chargers and operators; the only internet-facing component.

The API reaches chargers **through the gateway** (the "bridge"): commands over an authenticated internal HTTP call, liveness mirrored every few seconds, live events relayed through PostgreSQL `NOTIFY`. Both processes must share the same `INTERNAL_API_TOKEN`.

A single-process mode (`dist/apps/all-in-one.js`) exists for development and very small sites. This guide covers the recommended two-process production layout.

2. Architecture



3. Prerequisites

Item	Requirement
------	-------------

Server	Ubuntu 22.04 / 24.04 LTS (or equivalent), 2 vCPU, 4 GB RAM, 40 GB SSD minimum; 4 vCPU / 8 GB for more than ~300 chargers. Timezone <code>Asia/Jakarta</code> .
Path A	Docker Engine 24+ and the Docker Compose plugin.
Path B	Node.js 22 LTS, PostgreSQL 16, build tools (<code>build-essential</code>).
Both	Caddy 2.7+, <code>openssl</code> , <code>jq</code> (for log reading).
DNS	Two hostnames pointing at the server: one for chargers (e.g. <code>ocpp.example.co.id</code>) and one for the console (e.g. <code>console.example.co.id</code>).
Network	Inbound TCP 443 (and 80 for certificate issuance). Nothing else inbound. Outbound 443 for Let's Encrypt and, if used, the OpenStreetMap tiles in the operators' browsers.
Optional	HashiCorp Vault with a PKI engine, for Security Profile 3 client certificates. Not needed: PlugSure's own charging-station CA issues them.

Decide the charger hostname before any charger is configured. It is written into every charger's Central System URL; changing it later means reconfiguring every unit in the field.

4. Prepare the configuration

4.1 Generate secrets

Run once, store the values in your password manager, and never commit them:

```
openssl rand -hex 32 # AUDIT_HMAC_KEY      - keys the tamper-evident audit log
openssl rand -hex 32 # SECRETS_KEY         - encrypts provider credentials at rest
openssl rand -hex 32 # INTERNAL_API_TOKEN  - API ↔ gateway bridge (same value on both)
openssl rand -base64 24 # POSTGRES_APP_PASSWORD - runtime DB role plugsure_app
openssl rand -base64 24
# owner DB password (POSTGRES_PASSWORD on Path A, plugsure role on Path B)
```

AUDIT_HMAC_KEY cannot be changed casually. Rotating it makes every existing audit entry fail verification. Export and re-anchor the audit log first.

4.2 Production values

Variable	Production value	Why
NODE_ENV	production	Enables every production safety check.
OCPP_MIN_SECURITY_PROFILE	2	Chargers must use TLS + a per-unit key (or Profile 3 mTLS).
OCPP_TRUST_PROXY_PROTO	true	TLS terminates at Caddy; the gateway learns it from Caddy's header.
OCPP_AUTO_ADOPT	false	Unknown chargers wait for an operator instead of enrolling themselves.
OCPP_VERSIONS	ocpp1.6,ocpp2.0.1	Dual stack. Use ocpp1.6 alone until 2.0.1 hardware is certified. Add ocpp2.1 for OCPP 2.1 stations (bidirectional charging).
PUBLIC_BASE_URL	https://ocpp.example.co.id	Firmware download and log-upload URLs handed to chargers.
OCPP_PUBLIC_URL	wss://ocpp.example.co.id/ocpp	Printed on commissioning exports and QR codes.
INTERNAL_API_TOKEN	(secret)	Required. Without it the console cannot reach chargers.
API_TRUSTED_PROXIES	Path A: 172.16.0.0/12 Path B: 127.0.0.1	Lets the API see the real client IP and HTTPS scheme behind Caddy (rate limiting, audit IPs, Secure cookie).
TZ	Asia/Jakarta	Billing and ToU (WBP/LWBP) boundaries.

The full list is in section 15.

5. Install — Path A: Docker Compose (recommended)

1. Copy and extract the package:

```
sudo mkdir -p /opt && cd /opt
sudo unzip ~/plugsure-v1.3.0.zip          # creates /opt/plugsure
cd /opt/plugsure
sudo cp .env.example .env && sudo chmod 600 .env
```

2. Edit `/opt/plugsure/.env` and set at least:

```
NODE_ENV=production
TZ=Asia/Jakarta
POSTGRES_PASSWORD=<owner password>
POSTGRES_APP_PASSWORD=<runtime password>
AUDIT_HMAC_KEY=<hex>
SECRETS_KEY=<hex>
INTERNAL_API_TOKEN=<hex>
OCPP_MIN_SECURITY_PROFILE=2
OCPP_TRUST_PROXY_PROTO=true
OCPP_AUTO_ADOPT=false
OCPP_VERSIONS=ocpp1.6,ocpp2.0.1
PUBLIC_BASE_URL=https://ocpp.example.co.id
OCPP_PUBLIC_URL=wss://ocpp.example.co.id/ocpp
API_TRUSTED_PROXIES=172.16.0.0/12
```

The compose file already sets `GATEWAY_INTERNAL_URL=http://gateway:9220`, `EVENT_RELAY=true` and the storage volume.

3. Build and start (PostgreSQL → migrations → gateway + API, in that order):

```
sudo docker compose build
sudo docker compose up -d
sudo docker compose ps          # migrate: exited (0); postgres, gateway, api:
healthy
sudo docker compose logs --tail=50 api # expect "API bridge engaged" and "API + console
listening"
sudo docker compose logs --tail=50 gateway # expect "OCPP gateway listening", "background
workers started"
```

Persistent data lives in two named volumes: `plugsure-pgdata` (database) and `plugsure-storage` (firmware images, charger logs). Back both up (section 12).

6. Install — Path B: systemd (no Docker)

1. Create the service user and install the code:

```
sudo useradd --system --home /opt/plugsure --shell /usr/sbin/nologin plugsure
sudo mkdir -p /opt && cd /opt && sudo unzip ~/plugsure-v1.3.0.zip
cd /opt/plugsure
sudo npm ci
sudo npm run build
sudo cp -R src/web dist/web && sudo cp -R src/driver-web dist/driver-web # REQUIRED: the
console is served from dist/web
sudo chown -R plugsure:plugsure /opt/plugsure
sudo mkdir -p /var/lib/plugsure/storage && sudo chown -R plugsure:plugsure /var/lib/plugsure
```

2. Create the database. The owner role needs `CREATEROLE` (the migrations create the restricted runtime role `plugsure_app`); it must **not** be a superuser:

```
sudo -u postgres createuser plugsure --createrole --pwprompt
sudo -u postgres createdb plugsure -O plugsure
```

Server that already has a `plugsure_app` role (shared cluster, rebuild, or an earlier install): PostgreSQL 16+ only lets the owner alter a role it administers. Grant that once, or migration 006 fails with *permission denied to alter role*:

```
sudo -u postgres psql -c "GRANT plugsure_app TO plugsure WITH ADMIN OPTION;"
```

3. Create `/etc/plugsure/plugsure.env` (mode 600, owner root) — shared by both units:

```
NODE_ENV=production
TZ=Asia/Jakarta
# the API and gateway run as the restricted role (row-level security enforced)
DATABASE_URL=postgresql://plugsure_app:<APP_PW>@127.0.0.1:5432/plugsure
# migrations run as the owner (ExecStartPre of plugsure-api.service)
MIGRATION_DATABASE_URL=postgresql://plugsure:<OWNER_PW>@127.0.0.1:5432/plugsure
POSTGRES_APP_USER=plugsure_app
POSTGRES_APP_PASSWORD=<APP_PW>
AUDIT_HMAC_KEY=<hex>
SECRETS_KEY=<hex>
INTERNAL_API_TOKEN=<hex>
GATEWAY_INTERNAL_URL=http://127.0.0.1:9220
EVENT_RELAY=true
RUN_WORKERS=true
API_HOST=127.0.0.1
API_PORT=9200
OCPD_PORT=9220
API_TRUSTED_PROXIES=127.0.0.1
OCPD_MIN_SECURITY_PROFILE=2
OCPD_TRUST_PROXY_PROTO=true
OCPD_AUTO_ADOPT=false
OCPD_VERSIONS=ocpp1.6,ocpp2.0.1
PUBLIC_BASE_URL=https://ocpp.example.co.id
OCPD_PUBLIC_URL=wss://ocpp.example.co.id/ocpp
```

If a password contains @ : / ? # or %, percent-encode it inside the URL (e.g. @ → %40).

4. Install and start the units (the API migrates the schema before it starts; the gateway starts after it):

```
sudo install -m 0644 deploy/plugsure-api.service deploy/plugsure-gateway.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now plugsure-api plugsure-gateway
journalctl -u plugsure-api -n 50 --no-pager      # "migrations up to date", "API bridge engaged"
journalctl -u plugsure-gateway -n 50 --no-pager  # "row-level security is in force", "background workers started"
```

7. Caddy, DNS and firewall

1. Point the DNS names at the server's public IP and wait for propagation: the charger host, the console host and, for roaming, the `ocpi.` host.
2. Install `deploy/Caddyfile` as `/etc/caddy/Caddyfile`, then edit:
 - replace `ocpp.example.id` with your charger hostname, `console.example.id` with your console hostname and `ocpi.example.id` with your roaming hostname (or delete that block if you do not roam);
 - replace the allow-listed ranges (`203.0.113.0/24 10.0.0.0/8`) with your office / VPN ranges;
 - set the ACME contact e-mail.

```
sudo caddy validate --config /etc/caddy/Caddyfile
sudo systemctl reload caddy
```

The charger host serves `/ocpp/*` (→ gateway) and `/fw/*`, `/diag/*` (→ API, used by chargers for firmware and logs). The console host serves the console and API behind the allow-list. The roaming host serves only `/ocpi/*`, with no allow-list: partners authenticate with their credentials token. The optional `api.` host serves `/v1/*`, `/openapi.json` and `/api-docs.html` to integrators. It strips cookies and blocks the console sign-in routes, so only API keys (including sandbox keys) work there. Replace `api.example.id` with your hostname, or delete the block to keep the API console-only.

3. Firewall — only 22 (restricted), 80 and 443 inbound:

```
sudo ufw default deny incoming
sudo ufw allow from <admin-ip> to any port 22 proto tcp
sudo ufw allow 80,443/tcp
sudo ufw enable
```

Never expose 9200, 9220 or 5432 directly. The gateway's `/internal/*` endpoints are protected by the token, but should also be unreachable from outside.

8. Create the first administrator

This is the only command-line step in normal operation; every other user is invited from **Users & Roles** in the console.

```
# Path A
sudo docker compose run --rm api node dist/db/create-admin.js \
  --email ops.lead@your-cpo.co.id --name "Ops Lead" \
  --org-slug your-cpo --org-name "PT Your CPO Indonesia"

# Path B
cd /opt/plugsure && sudo -u plugsure bash -c 'set -a; . /etc/plugsure/plugsure.env; set +a; \
  node dist/db/create-admin.js --email ops.lead@your-cpo.co.id --name "Ops Lead" \
  --org-slug your-cpo --org-name "PT Your CPO Indonesia"'
```

The command prints a **one-time password**. Sign in at <https://console.example.co.id/>; you will be asked to choose your own password (12+ characters, mixing three of: lower case, upper case, digits, symbols). Then complete the organisation details (NPWP, PKP status) and invite your team:

Role	For
Super Administrator	Platform owners — everything, including users.
CPO Operations Manager	Runs the network; can view but not change users.
Site Host / Landlord	Sees only their assigned site(s): status, revenue, power. No commands.
Field Technician	Monitoring, diagnostics, configuration; remote start only with technician/VIP test cards.
Financial Auditor	Read-only with full session/invoice export and audit log.

9. Verify the installation

```
curl -s http://127.0.0.1:9200/healthz | jq      # {"ok":true,"db":true,...}
curl -s http://127.0.0.1:9220/healthz | jq    # gateway: {"ok":true,...}
curl -sI https://ocpp.example.co.id/ocpp/test # 426 Upgrade Required = TLS edge + gateway
reachable
curl -sI https://console.example.co.id/      # 200 from the allow-listed network, 403
elsewhere
```

In the console, confirm: the top-right **live** indicator is green; **Dashboard** loads; the sidebar footer shows `v1.3.0 · production`.

Automated acceptance run (staging only). `tools/e2e/console-e2e.mts` drives 96 checks across all modules with the bundled charger simulator. It creates test sites, chargers and users — **never run it against production**. See the header of that file.

10. Commission the first charger

1. **Sites** → **Create site**. Enter the PLN group, subscribed kVA and power factor (the card shows the $\text{kVA} \times \text{PF}$ ceiling, the 200 kVA TR/TM cliff and the reckoning minimum), SPKLU ID and scheme, SLO certificate and the PBJT-TL rate of the kabupaten/kota.
2. **Load management**. Confirm the ceiling (it cannot exceed $\text{kVA} \times \text{PF}$), set the auxiliary reserve, choose fair-share or priority.
3. **Tariffs** → **Create tariff plan** and assign it to the site (optionally AC-only / DC-only). Prices above the PLN layanan-khusus ceiling or the Kepmen ESDM service-fee caps cannot be saved.
4. **Onboarding** → **Add charge point**. The wizard walks through identity and hardware (manufacturer, model, serial by camera scan, firmware, OCPP version), site, security, EVSE/connectors (with tera data) and review.
 - **Profile 2**: it generates the key and shows a **QR code and .json commissioning file** — **shown once**. Tick **move it to Profile 3 automatically** to have the charger get its certificate over OCPP after its first connection.
 - **Profile 3: Issue automatically** makes the key and certificate. Download `client.key`, `client.crt` and `ca.pem` **once** and load them with the charger's commissioning tool. Alternatively, paste the charger's CSR so its key never leaves it.
5. Configure the charger with the Central System URL `wss://ocpp.example.co.id/ocpp/<IDENTITY>` and the key, then power-cycle it. The wizard's final step turns green: **Hardware Connected & Adopted**.
6. **RFID & Access** → **Push to all chargers at site** so registered cards work during WAN outages.

Security order matters: the key must be on the charger before the profile is raised. The wizard does this in the right order; if you change keys later from the Security tab, set the new key on the charger first. The previous key keeps working for 24 hours.

Security Profile 3 (client certificates). Put PlugSure's charging-station CA on the TLS terminator first: Onboarding → Certificate authority → download `plugsure-charger-ca.pem` to `/etc/caddy/`, uncomment the `tls { client_auth ... }` block in `deploy/Caddyfile` and reload Caddy. Keep its `header_up X-Client-Cert-Fingerprint` line: it stops a charger forging another's certificate. Certificates are renewed over OCPP before they expire; Onboarding → Certificates shows them all.

If the charger does not appear: open **Connections**. It lists every handshake with the reason it was refused (unknown identity, wrong key, no TLS, wrong OCPP version) in plain language.

11. Upgrading from v1.2.1

1. **Back up** the database (and, from now on, the storage volume):

```
pg_dump "$OWNER_DATABASE_URL" -Fc -f /var/backups/plugsure-pre-1.3.0-$(date +%F).dump
```

2. Keep your existing `.env` / `plugsure.env` and **add**: `INTERNAL_API_TOKEN`, `PUBLIC_BASE_URL`, `OCPP_PUBLIC_URL`, `API_TRUSTED_PROXIES`; on Path B also `GATEWAY_INTERNAL_URL`, `EVENT_RELAY=true`, `MIGRATION_DATABASE_URL`, and ensure the owner role has `CREATEROLE` (`ALTER ROLE plugsure CREATEROLE;`).

3. Replace the code with the v1.3.0 package (keep `.env`), then:

- Path A: `docker compose build && docker compose up -d` (migrations 009 to 042 run automatically);
- Path B: `npm ci && npm run build`, copy `src/web` and `src/driver-web` into `dist/`, reinstall `deploy/plugsure-api.service`, `systemctl daemon-reload`, restart both units.

4. Create the first administrator (section 8) — the old console had no sign-in.

5. Update Caddy from the new `deploy/Caddyfile` (adds `/fw`, `/diag`, the console host and the roaming host).

Migrations 009 to 042 are additive (013 adds commission statements, every site starting as "public"; 014 adds site owners, none assigned until you do; 015 to 017 add roaming, with nothing shared until you publish a site or share a card; 018 adds the driver app's favourites, push notifications and reservations; 019 adds developer sandboxes, none until one is created; 020 adds fleet billing, linking existing fleet cards to one account per fleet name, with nothing invoiced until you issue; 021 adds promotions and memberships, none until one is created; 022 adds Plug & Charge, off until switched on; 023 adds charger certificates, nothing changes until one is issued; 024 adds console-managed integrations, environment variables apply until something is set; 025 adds e-wallet and card payments, QRIS only until an operator enables more; 026 adds card holds and saved cards, off until an operator enables them; 027 adds linked e-wallets, off until an operator enables linking; 028 lets drivers pay an expired card hold in the app, used only when a hold expires; 029 adds WhatsApp delivery status, SMS, on-call rotas and per-site offline thresholds, none of which does anything until set up; 030 adds fleet credit notes and the fleet customer portal, with nothing credited and no portal user until you create them; 031 adds pass renewal, proration and loyalty points, with nothing renewing and nobody earning points until switched on; 032 adds the OCPP 2.0.1 device model, empty until a station reports; 033 adds the driver queue, off at every site until switched on; 034 adds reserving partner chargers, unused until a fleet driver reserves one; 035 adds reservation fees, none until you set one on a site; 036 adds partners' charging limits and hub client lists, empty until a partner sends them; 037 adds per-key API rate limits and usage, every key keeping the default; 038 adds ISO 15118 charging needs and bidirectional charging, off until a site is enabled; 039 adds signed meter values, recorded and verified but never blocking a session until a site is set to require them; 040 adds white-label driver apps, none until an operator creates one; 041 adds native iOS notifications, unused until an operator uploads an APNs key; 042 adds Live Activities, unused until an iOS app registers one): existing tariffs keep billing identically, existing connectors are treated as "tera verified", and nothing changes for chargers until an operator changes something. 010 adds the driver-app fleet PIN lockout counters; 011 adds refund tracking, charger outage history and the webhook outbox. 012 adds alert routing (e-mail and WhatsApp) and marks existing alerts as already routed, so nothing is sent for history.

Behaviour changes to brief operators on:

- A power ceiling above $kVA \times PF$ is now refused instead of silently clamped.
- Connectors marked "pending calibration" cannot sell energy.
- Unknown chargers are no longer auto-adopted outside development.
- A prepaid (QRIS) driver must start within 30 minutes of paying. After that the QR stops working and the payment appears under **Refunds**.
- Right after the upgrade, expect one critical "offline" alert for each charger that was already offline. Decommission units that are gone for good.

12. Operations

Backups

```
# nightly - database
pg_dump "$OWNER_DATABASE_URL" -Fc -f /var/backups/plugsure-$(date +%F).dump
# nightly - firmware images and diagnostic logs
tar czf /var/backups/plugsure-storage-$(date +%F).tgz /var/lib/plugsure/storage # Path B
docker run --rm -v plugsure-storage:/s -v /var/backups:/b alpine tar czf /b/plugsure-storage-$(date +%F).tgz -C /s . # Path A
```

Copy backups off the server. Test a restore every quarter.

Logs

```
docker compose logs -f gateway | journalctl -u plugsure-gateway -f
docker compose logs --since 30m api | jq -c 'select(.level >= 40)' # warnings and errors
```

Every OCPP frame is also stored and searchable in the console (**OCPP log**, with a live tail and NDJSON export). Every operator action is in **Audit log** with hash-chain verification.

Alert notifications (e-mail, WhatsApp and SMS)

Set up in the console under **Govern** → **Alert routing**, by a Super Admin or CPO Operations Manager. The gateway sends the messages, so it needs outbound access to your SMTP server (port 587 or 465), to `graph.facebook.com:443` and to your SMS provider (`api.twilio.com:443`, `console.zenziva.net:443` or your gateway). Set `CONSOLE_PUBLIC_URL` so each message links to the console.

1. **E-mail**. Enter the SMTP server, port, security (STARTTLS on 587 or TLS on 465), user name and password, and a sender address on a domain whose SPF/DKIM allows that server (otherwise alerts land in spam). For Google Workspace use `smtp.gmail.com`, 587, STARTTLS and an app password. Press **Send a test**.
2. **WhatsApp**. You need a Meta Business account with a WhatsApp Business sender number.
 - In WhatsApp Manager create a **Utility** template named `plugsure_alert` (language Indonesian, `id`) with this body:

```
Peringatan PlugSure ({{1}}): {{2}} Waktu: {{3}}
```

Here `{{1}}` is the status (CRITICAL, WARNING, RESOLVED, ESCALATED ...), `{{2}}` the alert, site and console link, and `{{3}}` the time (WIB). Wait for approval.

- Create a System User with the `whatsapp_business_messaging` permission and generate a **permanent** token.
- In the console enter the **Phone number ID** (API setup page, not the phone number), the token, the template name and language. Press **Send a test** to your own number.
- **Delivery status**. Open the channel's **Edit settings** again: it now shows a **callback URL** (`PUBLIC_BASE_URL/hooks/whatsapp/<key>`) and a **verify token**. In the Meta app go to WhatsApp → Configuration → Webhook, enter both, press *Verify and save*, and subscribe to the `messages` field. Then copy the **App secret** (App settings → Basic) into the channel's App secret field and save. Status callbacks without a valid `X-Hub-Signature-256` are refused. The callback URL must be reachable from the internet: the supplied Caddyfile forwards `/hooks/*` on the public OCPP host, next to `/pay/*`.

3. **SMS** (optional; for people without WhatsApp, or as a fallback). Choose the provider:

- *Twilio*: Account SID, auth token, and a sender number or a Messaging Service SID. For Indonesia register an alphanumeric sender through a Messaging Service; unregistered senders are often filtered.
- *Zenziva*: user key and pass key (the regular SMS API).
- *Your own gateway*: a URL that accepts POST
`{"to":"+62...","message":"...","channel":"sms","purpose":"alert"}` with the token as a bearer token.

Press **Send a test**.

4. **Contacts**. Add the people on call, with e-mail, WhatsApp number (any spelling, e.g. 0812...) and, only if it differs, an SMS number.

5. **On-call rotas** (optional). Under **On-call rotas**, create one per team: the people in shift order, daily or weekly, the handover time (in `ALERT_TIMEZONE`) and the first day. Use **Cover a shift** for leave and swaps. Rules can then notify "whoever is on duty".

6. **Rules**. A good start:

- *Critical* → *whoever is on duty*: WhatsApp with "send an SMS instead when WhatsApp fails", all sites, "resolved" messages on, escalate to the operations manager after 30 minutes.
- *Warnings* → *office*: e-mail, quiet hours 21:00–07:00.
- *Compliance (all types)* → *compliance officer*: e-mail.

7. Whoever takes an alert presses **Acknowledge** on the dashboard; that stops escalation. When the charger recovers the alert resolves itself and a "resolved" message goes out.

Every message is listed in **Alert routing** → **Delivery log**, with the provider's reason when one fails. With the Meta webhook set up, WhatsApp messages also show *delivered*, *read* or *failed*; without it "sent" only means accepted by WhatsApp, so check each contact's number once with **Send a test**. An SMS sent as a fallback is marked as such.

Offline alerts per site. A charger offline longer than `OFFLINE_ALERT_MINUTES` (default 15) raises a critical alert. A site with a weak mobile signal can have its own threshold: **Sites** → **edit** → **Monitoring** → **Offline alert after** (1–1440 minutes).

Platform commission and fee statements

Each customer organisation gets a monthly statement of what it owes the platform:

- commission on public charging, with the session subtotal (energy, service, admin and idle fees, **excluding PBJT-TL and PPN**) as the base
- the minimum per charger
- flat fees for private chargers

The default rates are the published ones (8% / 6.5% / 5% by site, minimum per charger Rp 150,000 AC / Rp 350,000 DC, private Rp 250,000 AC / Rp 450,000 DC).

1. Create the operator's billing account in the **operator's own** organisation, never in a customer's:

```
npm run create-admin -- --email billing@plugsure.com --org-slug plugsure --org-name "PlugSure" --platform-admin
```

2. Signed in with it, open **Govern** → **Platform billing**. For each customer:

- set the plan if it differs from the published rates; a plan takes effect from a chosen month

- mark fleet depots and staff or residential sites **Private**; every site starts as Public
- 3. Monthly, after the month ends: review each draft (notes flag, for example, a private site where drivers paid, or sessions awaiting review), then **Finalise month**. That freezes and numbers the statement (PSC-YYYYMM-...). Issue the faktur pajak separately.
- 4. Customers see their statements under **Commercial** → **Statements**, with CSV and a printable copy. They need org-wide invoice access.

Site owners: owner portal and billing

For sites that belong to other businesses (hotels, retail, offices) and that you operate.

1. **Commercial** → **Owners** → **Add owner**: name, legal name and NPWP (printed on statements), contact. Open the owner and tick its **sites**.
2. **Invite a portal user for this owner**, from the owner or from Users & Roles with the role *Site Owner (portal)*. Share the one-time password securely. The owner signs in at the console address with e-mail and password and sees only its own chargers, sessions, availability and monthly statement. It is read-only and follows the owner's sites automatically; archiving the owner removes access at once. It works on a phone or tablet; the owner can pick light or dark mode from the user menu (by default it follows the device).
3. **Commercial** → **Billing** shows every owner's month: kWh, sessions, gross, taxes, commission base, the owner's share and your share, plus your own sites, with totals and CSV. Open an owner to see its statement and to set its plan (from a chosen month). After the month ends, **Finalise month**.
4. Keep *Seller on driver receipts* as the operator until driver payments are settled to the owner's own merchant account; then switch it, so receipts carry the owner's legal name and NPWP.

A site that already has charging history with one owner cannot be given to another (the new owner would see the old owner's revenue). Create a new site for the new owner and move the chargers; their earlier OCPP log stays hidden from the new owner.

Bidirectional charging (V2G / V2B, ISO 15118-20)

Cars that support bidirectional charging can give energy back to a site while its load peaks. Off everywhere by default.

1. **Chargers**. Only OCPP 2.1 stations can be asked to discharge. Add `ocpp2.1` to `OCPP_VERSIONS` on the gateway and register the charger as OCPP 2.1. Test each charger model with each car on the bench, including ISO 15118-20 dynamic control.
2. **The site's own load**. Under Load management, set the auxiliary load (air-conditioning, lighting, the shop). Without a PLN export agreement, discharge is capped at it, so energy never flows back to the grid.
3. **The programme**. Under Sites → Edit site → Bidirectional charging, set:
 - the hours, e.g. the 17:00–22:00 evening peak;
 - a site limit, if wanted;
 - the battery floor, 40 % by default;
 - the driver's credit per kWh, taken off their session before tax.

Tick *Energy may flow back to the PLN grid* only with a PLN export agreement.

4. **Consent**. App drivers are offered it on the live-charge screen when their car can give energy back. Fleets agree for their cards under Fleet billing → account. Nothing discharges without consent, below the floor, or in the hour before the driver leaves.

5. **Watch.** A session's **Car (ISO 15118)** tab shows what the car asked for, its battery level, energy given back and the credit, and whether it is discharging, or why not.

Agree the tax treatment of the credit with your tax adviser. PlugSure treats it as a reduction of the session before PBJT-TL and PPN.

Not included yet: grid-support functions (frequency and voltage support, DER control) and price schedules sent to the car wait for a PLN or DSO grid code for EV discharge. Exported energy is not sold on a market, because PLN is the single buyer, so the credit is your own. AC bidirectional (AC_BPT) runs in the sandbox but has not been tried on AC hardware: bench-test it first.

White-label driver apps

An operator can give its drivers its own app: the same driver app, under the operator's name, colours and icon, on its own web address, showing only its stations. Set it up under **Commercial** → **Driver app**.

1. Name and look. Enter:

- the app name, a short name and a tagline;
- one accent colour, which PlugSure adjusts per theme so text in it always reads at 4.5:1;
- an icon background;
- support contacts and the privacy policy.

Upload one square PNG icon, 1024 px if possible. The phone-sized preview beside the form is the real app.

2. Web address. Choose one, e.g. `app.operator.co.id`, and point it at the driver host with a DNS `CNAME`. Save it and set the status to **Live**. Caddy issues the certificate on the first HTTPS request, after checking with PlugSure that the name belongs to a driver app. This needs the `on_demand_tls` option and the catch-all block from the supplied Caddyfile.

3. iOS notifications. In the Apple Developer account, go to Certificates, Identifiers & Profiles → Keys → + and tick *Apple Push Notifications service (APNs)*. Download the `.p8`; Apple lets you do this once. Upload it with its Key ID under **iOS notifications**. PlugSure checks it with Apple straight away without notifying anyone, and shows how many iPhones are registered. The app icon then shows how many sessions wait to be paid, and notifications carry buttons (stop charging, view receipt, pay now, give up my turn, cancel reservation) and, when charging finishes, a picture of the charge. For the buttons and the picture, follow the two Xcode steps in the build kit's README. **Live Activities** (iOS 16.2+) show a charge under way on the lock screen and in the Dynamic Island: add the kit's widget extension and plugin (README step 6). The same key sends their updates; the gateway checks every 5 s and sends at most every 30 s (`LIVE_ACTIVITY_MIN_INTERVAL_S`). The gateway needs outbound HTTPS to `api.push.apple.com` and `api.sandbox.push.apple.com`.

4. Store apps. Fill in:

- the Android package name;
- the signing-certificate SHA-256 fingerprints (the upload key, and later Play's app-signing key);
- the iOS bundle identifier and Apple Team ID;
- the version.

Then download the **Build kit** and follow its README:

- Android: a Trusted Web Activity built with Bubblewrap, or the included GitHub Actions workflow;
- iOS: a Capacitor shell built in Xcode on a Mac.

Raise the version code for every upload. The web address serves the files that tie the store apps to it (`/.well-known/assetlinks.json`, `/.well-known/apple-app-site-association`).

The store accounts, signing keys and builds are the operator's.

- Keep the Android upload keystore safe: without it the app cannot be updated.
- Apple reviews web-based apps strictly (guideline 4.2). Describe the charging features in the review notes and give a test account.

Signed meter values (OCMF)

Calibration-law meters sign their register readings. PlugSure verifies each signature against the meter's public key, then checks the signed start and end readings against the energy it bills. Drivers and auditors can check the result. Nothing changes for chargers that do not sign.

1. **Register the meter.** In Onboard, enter each connector's **meter serial** and **meter public key**, from the meter's label or the vendor. Hex, base64, PEM and the raw 64/65-byte point are all accepted. Without a registered key, readings are only "unverified key": PlugSure never trusts a key sent by the charger itself.
2. **Switch signing on at the charger.** For OCPP 2.0.1 / 2.1, use `POST /v1/charge-points/<identity>/signed-metering` with `{"enabled":true}`. It sets SignReadings, and sends the public key once per transaction. For OCPP 1.6, use the vendor's configuration key.
3. **Choose the policy.** Under Sites → Edit site → Signed meter data:
 - *Record* (default): keep, verify and warn on a mismatch;
 - *Require*: sessions that are not verified are parked for review and not invoiced;
 - *Off*.
4. **Check.** A session's **Signed meter data** tab shows the outcome, the readings and the OCMF text, with an XML download for the Transparency Software. The tax receipt and the driver app show the status, and OCPI CDRs carry the signed data.

Statuses: **verified**; **unverified key** (valid only against the charger's own key); **mismatch** (signed kWh differ from the bill by more than 2 Wh); **invalid** (a bad signature, another meter, or a key conflict); **incomplete** (start or end missing); **missing** (required, none sent). Only OCMF is verified; other formats are kept as "unsupported".

Roaming (OCPI 2.2.1)

Two directions, on one page (Commercial → Roaming):

- Drivers of other networks charge on **your** chargers. Their provider (an eMSP, directly or through a roaming hub) receives a charge record (CDR) for every session and pays you. Your prices, taxes and receipts are unchanged.
 - Your RFID and fleet cards charge on **other operators'** chargers. The operator (CPO) sends you a CDR for every session; you pay it and bill your card holder.
1. **Expose the roaming host.** Keep the `ocpi.example.id` block of `deploy/Caddyfile` (section 7) and set `OCPI_PUBLIC_URL=https://ocpi.example.id` on the API and the gateway. Partners must reach `/ocpi/*` from the internet; the console host's allow-list would block them.
 2. **Commercial → Roaming → Set roaming identity:** country code `ID` and a three-character party ID agreed with your hub or partners (e.g. `PLS`). Partners cannot follow a change later.

3. **Add partner**, either way round:

- *The partner connects to us*: send them the versions URL and the one-time token shown (it stops working once they register);
- *We connect to them*: enter their versions URL and token. PlugSure picks OCPI 2.2.1 and registers.

The partner shows as **connected** when done.

4. **Shared sites**: share the sites partners may show. A site needs a map location, a street address and a city (Sites → Edit); private sites cannot be shared. Changes reach partners within a minute; status changes within seconds.
5. **Watch**: each partner's message log and "sent to partner" list, and **Roaming sessions** (was each CDR accepted?). A CDR a partner does not accept raises an alert; fix the connection, then **Send failed again**.
6. **Your cards on other networks**: add the operator as a partner of type *Charge point operator*. Its chargers appear under **Partner network**. Under **Cards abroad**, allow roaming per card (or **Share all active cards**). Cards with an energy or spending limit are checked with PlugSure before every session, and roaming charges count towards the limit. Blocking a card in the RFID centre stops it on every network within seconds. Fleet drivers with such a card see the partner chargers in the driver app (**Jaringan mitra**), can start and stop charges there, and get the operator's charge record in their history.
7. **Bill on: Cards abroad** → **Export CSV** lists every charge record from other networks (where, card, holder, fleet, energy, amounts) for invoicing your fleets. Driver-app (QRIS) users cannot roam: they pay upfront and have nothing to bill afterwards.
8. **Partner smart charging**: a partner may limit how fast its driver's session charges (OCPI ChargingProfiles). The limit is applied through load management, on top of the site's power budget, so it can only slow a session down. Current limits are listed under **Roaming sessions**. Your chargers must accept transaction profiles (SetChargingProfile), as load management already requires.
9. **Hubs**: a hub reports the parties behind it (OCPI HubClientInfo). They are listed in its panel under **Behind this hub**, refreshed every 6 hours or with **Refresh from hub**. Once a hub has sent the list, it may act only for the parties on it that are connected or offline.

Before live traffic: run the partner's or hub's connection test against its test environment, agree settlement terms, and confirm the tax treatment. By default a CDR's `excl_vat` is the subtotal plus PBJT-TL and `incl_vat` is the receipt total (`OCPI_PBJT_IN_EXCL_VAT`).

Driver app: map, notifications and reservations

No setup is needed to switch these on. Things to know:

- **Map tiles**. The station map loads tiles from `MAP_TILE_URL`, which defaults to the OpenStreetMap Foundation's server. That server is for light use only. Before real traffic, point it at your own tile server or a commercial one (e.g. MapTiler, Stadia, Thunderforest), set the provider's `MAP_ATTRIBUTION`, and restart the API. The page's security policy allows the new tile host automatically. Sites need latitude and longitude (Sites → edit) to appear on the map.
- **Push notifications**. The first use generates a VAPID key pair and stores it in the database, sealed with `SECRETS_KEY`, so the API and gateway share it. Keep that key: changing it, or `SECRETS_KEY`, means every phone must switch notifications on again. The **gateway** sends notifications and needs outbound HTTPS to `fcm.googleapis.com`, `*.push.services.mozilla.com`, `web.push.apple.com` and `*.notify.windows.com`. Set `VAPID_SUBJECT` to a mailbox you read, because push services use it to contact senders.

- **Reservation fees.** Free by default. Set a fee per site under Sites → Edit site → Reservations. App drivers pay it before the connector is held; fleet cards are billed on the fleet invoice. It is kept once the connector is held, and refunded (Refunds) or left off the invoice when the charger refuses or the driver cancels within 2 minutes. The fleet invoice's e-Faktur uses the fee item code (Fleet billing → Settings) for these lines too.
- **Many stations.** Stations that would overlap on the map are grouped into a numbered bubble until the driver zooms in; nothing to configure.
- **Partner chargers.** Fleet drivers with a roaming card can reserve a partner operator's charger for 15 minutes. PlugSure sends the operator OCPI RESERVE_NOW , and the answers show in Commercial → Roaming → commands. Ask each partner whether it accepts reservations. One that does not answers NOT_SUPPORTED , and the driver is told.
- **iPhone.** Web Push works only when the driver has added the app to the Home Screen (iOS 16.4 or later); the app explains this under Akun.
- **Reservations.** The charger receives ReserveNow for 15 minutes (DRIVER_RESERVATION_MINUTES). Chargers that do not support reservations answer Rejected, and the driver is told so. Turn the feature off with DRIVER_RESERVATIONS=false . Check it once per charger model during acceptance: reserve, confirm the connector shows Reserved at the charger, then start with the same card or QR.

API reference and developer sandboxes

- **Reference.** /openapi.json is the OpenAPI 3.1 document and /api-docs.html the readable reference. Both are public, static and part of the build. After changing a route, run npm run openapi ; npm test fails if the document is out of date.
- **Sandboxes** are created under Govern → Developers, by a user with org:write . Each one is a separate tenant with two virtual chargers, a site, a tariff, three RFID cards and its own API key, shown once. The limit is 3 per operator. Hand the key to the integrator; rotate or delete the sandbox when the work is done.
- **Protocols.** A charger registered in a sandbox becomes a virtual charger speaking the version it is registered with: OCPP 1.6, 2.0.1 or 2.1. The gateway's OCPP_VERSIONS must list that version. Each virtual charger has a signing meter (OCMF), with its key registered automatically, so its sessions show as verified. On 2.1 the virtual car can give energy back (DC_BPT on DC, AC_BPT on AC), so bidirectional charging can be tried without hardware.
- **Virtual chargers run in the gateway** (the process with RUN_WORKERS=true). They are started within 10 seconds of creation and after every gateway restart. They use no network port and no extra configuration. A gateway restart interrupts their running sessions, as a power cut would; the stuck-session reconciliation closes them later.
- **Isolation.** Sandbox tenants never appear in the driver app, in roaming, in platform billing or in the operator's own lists. Their chargers cannot be reached from the network. Their records stay in the audit trail after deletion.
- **Load.** A sandbox charger costs about as much as a real one: a few database writes per meter sample. Three sandboxes add six chargers.
- **TypeScript SDK.** Download it from Govern → Developers → TypeScript SDK (/sdk/plugsure-csms-sdk.tgz) and give it to integrators. They install it with npm install ./plugsure-csms-sdk.tgz . After changing a route, run npm run openapi and then npm run sdk ; npm test fails if either is out of date. The /sdk/ path sits on the console host, behind its office allow-list.
- **Rate limits.** Each API key has its own limit: 600 requests a minute by default (API_KEY_RATE_LIMIT_PER_MIN). Set a key's own limit, and see its usage, under Users & roles → API keys. The console and other callers without a key are limited per IP (API_RATE_LIMIT_PER_MIN). Limits are kept per API process: two API processes behind a balancer double each key's allowance.

Fleet billing and e-Faktur

Fleet cards are billed monthly to their company's **fleet account** (Commercial → Fleet billing). One-time setup:

1. Settings:

- enter your NPWP (16 digits), NITKU and billing address, the invoice number prefix, and how to pay (bank account)
- for e-Faktur, enter the item type (goods A or services B), the 6-digit code and the Coretax unit code as your tax adviser classifies EV charging, then tick **confirmed**
- until then the XML export is refused

2. **Accounts:** every fleet name on your cards already has an account. Add each company's legal name, NPWP (or NIK), NITKU if a branch, billing e-mail and payment terms. Without an NPWP/NIK, no faktur can be prepared for that account.

3. **E-mail:** invoices go out through the e-mail channel under Govern → Alert routing. Set it up and send a test.

Each month, after it ends:

1. Open the month and check the drafts and their warnings.
2. Click **Issue all invoices**, then e-mail them. The e-mail carries the invoice as a **PDF** and the session list as CSV; the console downloads the same PDF.
3. Download **e-Faktur XML** and import it in Coretax (Faktur Pajak → Pajak Keluaran → Impor Data). Check the drafts there, then approve and sign them.
4. Record each faktur number back on its invoice.
5. Record payments as they arrive. Unsettled credit notes are listed under **Credit notes**: record each refund there once it is paid out.

Correcting an issued invoice. Open it and choose **Credit note**: the whole of what is left, or lines of an amount each (what the customer gets back, PPN included; a line with PPN is split into DPP and PPN like an invoice line). The invoice itself never changes.

- On an unpaid invoice the credit reduces what is owed, and settles the invoice when nothing is left.
- On a paid invoice it is refunded (record the refund when paid) or deducted from the account's next invoice.
- If the invoice's faktur pajak was already reported, record the matching *nota pembatalan* (or a replacement faktur) in Coretax. The console warns you. Credit notes are not in the XML export.
- A credit note issued in error can be voided while nothing has been done with it.

Fleet customer portal. In the fleet account, **Portal access** → **Invite a user** (needs user management), or Users & Roles with the role *Fleet customer (portal)*. Share the one-time password securely. The customer's staff sign in at the console address and see only that account: what it owes, its invoices and credit notes as PDF, this month so far, and its cards. They can block a lost card at once, and unblock only a card they blocked themselves; a card you blocked stays blocked. Archiving the fleet account removes their access.

Portal users are outside your office. The supplied Caddyfile keeps the console behind an office / VPN IP allow-list. Fleet customers (and site owners) need to reach it: publish the console on its own hostname without the allow-list for them, or add their addresses. Every portal route checks the user's own account, and portal users hold no operator permission.

Before the first real export: import one file into Coretax and compare it with the invoice. The file follows the DJP import template, which DJP revises from time to time. To correct an issued invoice before it is paid, you can also void it (the number is kept) and issue again; if its faktur was already approved, cancel or replace it in Coretax too. Once paid, use a credit note.

Integrations (payments, sign-in codes)

Connect the services drivers depend on under **Govern** → **Integrations**. **In production, QRIS checkout and phone sign-in are refused until these are connected.** Secrets are stored encrypted with `SECRETS_KEY` and never shown again; changing that key means entering them again.

1. **QRIS payments:** choose Midtrans, Xendit or your bank (BI-SNAP) and enter the credentials from the provider's dashboard. Each operator can connect its own merchant account; the platform's is the default.
 - Copy the **payment notification URL** shown after saving into the provider's dashboard. `/pay/*` must be public; `deploy/Caddyfile` routes it on the OCPP host.
 - Press **Test**, then make one real payment of a small amount and check it in the Activity list.
 - **Payment methods offered to drivers:** QRIS by default. Tick GoPay, ShopeePay and cards (Midtrans), or OVO, DANA, ShopeePay, LinkAja and cards (Xendit), only if they are activated on your acquirer account. Test one small payment with each method you enable.
 - **Card holds and saved cards** (optional, off by default): *hold, then charge only what is used* turns card payments for charging into pre-authorisations: the session's total is captured, the rest released. *Let signed-in drivers save a card* keeps the acquirer's token, never the card number; a card whose token has ended at the acquirer (deleted at the bank, or expired) is refused with a message naming it and no longer offered. Ask the acquirer to enable card pre-authorisation and card saving first, then test a hold end to end: hold, charge, capture, release. A hold that expires at the acquirer before capture (after about 7 days at Midtrans) is shown under Refunds as *expired, not charged* with the amount to collect, with a critical alert; it cannot be retried, and the driver can pay it from the receipt in the app (then *expired, paid in app*, and the alert resolves).
 - **Linked e-wallets** (optional, off by default): *let signed-in drivers link ... for one-tap payments* lets drivers link GoPay (Midtrans, or Xendit with GoPay recurring activated) or OVO / DANA / ShopeePay / LinkAja (Xendit) once and pay in one tap; the chosen amount is charged and unused balance refunded automatically. Ask the acquirer to enable e-wallet tokenisation, then test a link, a one-tap payment and its automatic refund.
 - **Post-pay** (optional, off by default): *linked e-wallets: charge after the session* starts the session with nothing charged and charges the rated total to the linked e-wallet afterwards, up to the per-session limit under Advanced (default Rp 200,000; larger amounts are charged up front). E-wallet balances (GoPay, OVO, DANA, ShopeePay, LinkAja) are checked before starting where the acquirer reports them (for GoPay at Midtrans: the wallet, or GoPay Tabungan, never PayLater credit; the GoPay token is looked up again before each charge, as Midtrans asks; a GoPay, OVO, DANA, ShopeePay or LinkAja link the driver ended in the e-wallet app, or that expired, is refused with "link ... again" instead of a balance message, and an unpaid session can be paid once it is linked again); *post-pay only when the e-wallet balance can be checked* (Advanced) charges the others up front. Unpaid charges are retried, can be paid by the driver from the receipt (by the e-wallet again, or in the app with any other method, also while the e-wallet PIN is pending or after it expired, was denied or was cancelled, with an unpaid notice on the home screen and Web Push reminders 15 minutes, 1 day and 3 days after the session, and a tax receipt issued as a nil transaction (Rp 0, "NIHIL / NIL") until paid; with Xendit, also point its payment method and payment token callbacks at the payment notification URL so ended links are known at

once; a payment that arrives after the session is paid is refunded), and are listed under **Refunds** →

Holds and post-pay.

2. **Driver sign-in codes:** WhatsApp Cloud API (an approved *Authentication* template with a copy-code button), Twilio, Zenziva or your own gateway. Add an SMS fallback, and test with your own number.
3. **Plug & Charge PKI** and **map tiles** are here too. The environment variables still work when nothing is set.

E-wallets and cards are pre-purchases like QRIS: the driver pays a fixed amount in the e-wallet app, or on the acquirer's hosted 3-D Secure card page, so card numbers never reach PlugSure. The acquirer returns the driver to `/app/paid.html` on `DRIVER_PUBLIC_URL` (else `CONSOLE_PUBLIC_URL`), and its signed notification confirms the payment. Unused balance is refunded by API where the acquirer supports it (Midtrans; Xendit for e-wallets); otherwise it is refunded by bank transfer from the Refunds page.

Card holds never need a refund: the capture is for what the session cost. A capture the acquirer refuses is retried automatically and listed under **Refunds** → **Card holds** with **Retry now**; after the last automatic try a critical alert is raised. An authorisation lapses at the acquirer (Midtrans: 7 days by default), so clear sessions parked for review before then.

Switching acquirer keeps the old account for the payments it took: its notification URL still works and its refunds still go to it. Leave the old URL in the old provider's dashboard until those payments have settled.

Promotions and memberships

Set up under Commercial → Promotions & plans (permission `tariff:write`). Discounts are applied at rating, before PBJT-TL and PPN, and printed as lines on the receipt. Each session gets the customer's membership plus at most one promotion, whichever is cheapest for the customer.

- **Plans:** a monthly fee with a member price per kWh, % off energy, included kWh and/or a waived service fee.
 - Fleet accounts or single cards are enrolled on the Members tab; their fee is charged on the monthly fleet invoice, for the days of the month the membership was in force (a whole month costs the whole fee). Set the membership-fee e-Faktur item (usually services, B) in Fleet billing → Settings and confirm it with your tax adviser.
 - Tick **offered in the app** to let drivers buy a 30-day pass (QRIS, e-wallet or card). Drivers are reminded 3 days before it ends.
 - **Automatic renewal:** a driver who pays with a saved card or linked e-wallet can let the pass renew itself; the gateway charges it a day before the end, retries a decline, and tells the driver when a PIN or 3-D Secure is needed or renewal had to stop. Saved cards and linked e-wallets must be on for the acquirer (Govern → Integrations). On Midtrans, unattended saved-card charges need 3-D Secure off for saved cards (`savedCard3ds: false`), otherwise every renewal waits for the driver.
 - **Switching plans:** the unused value of the current pass is credited: a dearer plan costs the difference, a cheaper one is free and runs longer.
- **Promotions:** for everyone, new drivers, chosen fleet accounts, plan members, or a promo code entered in the app. They can be limited by dates, days, a time window, sites, AC/DC, minimum kWh, total and per-customer uses, and a budget. Stop one by switching it off; sessions already rated keep their discount.

Loyalty points (Loyalty points tab, off by default): set the points per Rp 1,000, what a point is worth, the most points may pay of a session and when they expire, then switch it on. Drivers signed in to the app earn on every session; those who tick "use my points" get them off their next sessions automatically, before PBJT-TL

and PPN. The tab shows the points outstanding and their value in rupiah: **book that as a liability**, and brief finance before switching it on. Adjust a driver's points there (goodwill or corrections; audited).

Check a new promotion on a test session before announcing it. App passes are paid through the same acquirer as charging, so the live acquirer (P0) is needed before passes can be sold.

ISO 15118 Plug & Charge

A car with a charging contract identifies itself with its contract certificate; the driver just plugs in. PlugSure is the CSMS side. You also need a V2G PKI provider (Hsubject or similar) with a CPO agreement, and chargers with ISO 15118 Plug & Charge firmware.

1. **Connect the PKI.** Set `PNC_PKI=http`, `PNC_PKI_URL` and `PNC_PKI_TOKEN` on the API and the gateway. The URL is your PKI gateway, whose three calls are listed in `deploy/README.md` (Plug & Charge). Allow the gateway outbound HTTP/HTTPS to the PKI's OCSP responders.
2. **Trust anchors** (Operate → Plug & Charge): **Fetch from the PKI.** This gets the V2G root, and the mobility operator roots that contract chains are checked against.
3. **Chargers:** for each ISO 15118 charger, **Switch Plug & Charge on, Install trust anchors and Request V2G certificate.** Its certificate appears as installed with its expiry. It is renewed 30 days before it expires (`PNC_RENEW_DAYS`), and an alert is raised a week before.
4. **Contracts:** register your customers' eMAIDs, retail or billed to a fleet account. Partners' contracts arrive through roaming.
5. **Overview:** switch Plug & Charge on. Decide whether to accept a contract when its OCSP responder cannot be reached; the default is accept, since the charger has checked the chain.

Try each charger model with a test contract from your PKI before announcing Plug & Charge. The **Log** tab shows every exchange with its outcome. `PNC_PKI=mock` is a test PKI for development and sandboxes; production refuses it.

OCPP 2.0.1 stations: the device model

OCPP 2.0.1 stations are configured through their device model rather than 1.6 configuration keys. Open the charger and go to the **Device model** tab. Nothing needs setting up: migration 032 adds the tables.

1. **Request report** (full inventory). The station sends its components and variables in parts, and the tab shows them as they arrive.
2. Change a value and press **Save**. The console checks the value against the station's own limits and allowed values before it sends `SetVariables`. *RebootRequired* means it applies after the next reboot (Remote control → Reboot).
3. **Monitor...** on a variable that supports it adds a threshold, delta or periodic monitor. An alerting monitor raises a `charge_point.device_event` alert, routed like any other. It resolves when the station reports the event cleared.

Security settings (SecurityCtrlr) and the network connection profile are not changed from this tab. Use the Security tab and Onboarding, which keep the station from being locked out. Before rollout, request a full report from each 2.0.1 station model on the bench: vendors differ in which variables they report and which they allow to be changed.

Driver queue at busy sites

For sites where drivers regularly find every connector taken. Open the site, choose **Edit site** and tick **Driver queue**. Then set:

- **Time to start**: how long a freed connector is held for the next driver (2–15 minutes, default 5).
- **Longest queue**: the most drivers waiting at once (default 20).
- **Longest wait**: how long a driver can wait before their place ends (default 120 minutes).

How it works:

- Drivers join from the station page in the app. It is first come, first served.
- When a connector frees up, the charger holds it (ReserveNow) for the first driver waiting who can use it. Nobody else can take it meanwhile.
- A driver who does not start in time misses their turn, and the next driver gets the connector.

The site's **Driver queue** tab shows who is waiting and who holds a connector, with the last 24 hours. You can remove a driver there. Nothing to configure on the servers: offers are made by the gateway.

Check first that the site's chargers accept ReserveNow (reserve a connector from the app once). A charger that refuses it cannot hold connectors for the queue. Waiting drivers are still protected in the app, but a walk-up with an RFID card could take the connector at the charger.

Updates and rollback

Take a backup before every update. Roll back **code**, never the schema — migrations are forward-only:

```
# Path A  PLUGSURE_TAG=<previous-tag> docker compose up -d --no-deps gateway api
# Path B  restore the previous package into /opt/plugsure, rebuild, copy web assets, restart
both units
```

If a migration itself is the problem, restore the pre-upgrade dump.

Routine

- Daily: Dashboard alerts (tera lapsed / due, SLO expiry, faults, key rotation due).
- Weekly: Compliance page; Sessions → filter "review" and resolve.
- Daily (finance): **Refunds**. Pay what is due through the provider, or refund by bank transfer and record the reference. Drivers see "refund in progress" until you do.
- Monthly: **Availability**. Check uptime per charger (export CSV for site hosts and SLA reviews).
- Integrations: **Webhooks** pushes sessions, CDRs, charger status, alerts and refunds to your own systems. Receivers must verify the `PlugSure-Signature` header. Failed deliveries can be replayed from the endpoint's delivery log.
- Per rotation policy: rotate AuthorizationKeys from the charger's Security tab.

13. Troubleshooting

Symptom	Cause	Fix
Console shows every charger offline ; commands fail "not connected"	Bridge not configured	Same <code>INTERNAL_API_TOKEN</code> on both processes; <code>GATEWAY_INTERNAL_URL</code> on the API. API log must say "API bridge engaged".

Charger refused, Connections shows <code>rejected_tls_required</code>	Charger uses <code>ws://</code> while Profile ≥ 2	Configure <code>wss://</code> on the charger.
<code>rejected_auth</code>	Wrong or missing key	Re-enter the key on the charger, or rotate it from the Security tab.
<code>rejected_unknown_cp</code>	Identity not registered / typo	Onboard it, or use Connections → Pending adoption → Suggestions to spot the typo.
Gateway exits "requires TLS"	Profile 2 without TLS info	Set <code>OCPP_TRUST_PROXY_PROT0=true</code> (TLS at Caddy).
Process exits "AUDIT_HMAC_KEY" / "superuser"	Missing secret / wrong DB role	Set the secrets; the app must connect as <code>pluginsure_app</code> , never a superuser.
Migration fails "permission denied to create role"	Owner lacks CREATEROLE (Path B)	<code>ALTER ROLE pluginsure CREATEROLE;</code>
Migration 006 fails "permission denied to alter role"	<code>pluginsure_app</code> already existed, created by another role	<code>GRANT pluginsure_app TO pluginsure WITH ADMIN OPTION;</code>
Console 404 while <code>/healthz</code> works (Path B)	Web assets not copied	<code>cp -R src/web dist/web</code>
Sign-in works but you are logged out immediately	Secure cookie over plain HTTP	Use HTTPS via Caddy; set <code>API_TRUSTED_PROXIES</code> correctly.
Firmware campaign fails "PUBLIC_BASE_URL is not configured"	No public URL for downloads	Set <code>PUBLIC_BASE_URL</code> to the charger hostname.
Sign-in refused with "invalid email or password" although the password is right	5 failed sign-ins paused the account for 15 minutes. The message is the same for every failure on purpose, so it never reveals whether an address has an account.	Wait 15 minutes, or an administrator resets the password.
Webhook endpoint shows "failing" / "disabled"	Receiver down, slow (> 10 s), or not https	Fix the receiver, press Send test event , then Enable and Replay failed . An endpoint is disabled only after 50 straight failures with no success for 24 h.
Webhook deliveries fail "signing secret cannot be decrypted"	<code>SECRETS_KEY</code> was changed	Rotate the secret on each endpoint and update the receivers.
WhatsApp alert "failed" with code 132001 / "template does not exist"	Template name or language differs from the approved one, or not approved yet	Match the name and language code exactly (e.g. <code>id</code>); wait for approval.
WhatsApp "failed" with code 190 / "access token"	Temporary token expired	Use a permanent System User token.
WhatsApp "sent" but the person got nothing	Number not on WhatsApp, or not the right format	Send a test to that number; check the contact's number includes the country code.

Alert e-mails fail "Greeting never received" / time out	Outbound SMTP blocked (many clouds block port 25/587)	Open outbound 587/465 from the gateway, or use a provider on 2525/443.
Alert e-mails arrive in spam	Sender domain has no SPF/DKIM for the SMTP server	Use a sender address on a domain configured for that server.
No alert messages at all	Channel off, no matching rule, or workers off	Check the channel status and rules; one gateway must run with <code>RUN_WORKERS=true</code> .
No webhooks are sent at all	Workers off on every gateway	Run one gateway with <code>RUN_WORKERS=true</code> .
Offline alerts arrive late, all at once	The gateway did not run for a while (host sleep, suspended VM, blocked event loop): the log shows <code>gateway paused</code>	Keep the host awake and powered (no sleep, lid-close or battery-saver standby). <code>GATEWAY_DIAG=verbose</code> logs event-loop delay, worker pass times and database waits every 10 s.
Plug & Charge car refused; Log shows <code>CertChainError</code> or <code>unknown_contract</code>	MO root not installed / contract not registered	Trust anchors → Fetch from the PKI, then Install on the charger; register the eMAID under Contracts (or check the partner's roaming token).
Charger V2G certificate "signing failed"	PKI gateway unreachable or not configured	Check <code>PNC_PKI_URL</code> / <code>PNC_PKI_TOKEN</code> on the gateway process; Overview shows the PKI problem. Request the certificate again.
Profile 3 charger refused, Connections shows "Client certificate check failed"	CA not trusted by the TLS terminator, or another certificate on the charger	Install the CA from Onboarding → Certificate authority on Caddy (<code>client_auth</code>); check the certificate on the charger is the one issued (Onboarding → Certificates shows its serial).
Remote start refused "tera"	Meter lapsed or pending calibration	Update the connector's tera data (Connectors tab) after re-verification.

14. Go-live checklist

- ☐ Production values set: `NODE_ENV=production`, Profile 2, auto-adopt off, trust-proxy on
- ☐ All secrets generated, stored in the password manager, `.env` is mode 600
- ☐ Charger hostname final; TLS certificate issued (not the ACME staging CA)
- ☐ Ports 9200 / 9220 / 5432 not reachable from outside; console host IP allow-listed
- ☐ Logs show "row-level security is in force" and "API bridge engaged"
- ☐ First administrator created and password changed; team invited with least-privilege roles
- ☐ Organisation NPWP / PKP status confirmed with the finance team
- ☐ Every site has SPKLU ID, SLO and the correct PBJT-TL rate; tariffs assigned
- ☐ Tax treatment confirmed with a tax adviser (PBJT base, PBJT inside PPN base)
- ☐ Acceptance demo (`docs/ACCEPTANCE-v1.3.md`) passed on at least one real charger, including cable unlock < 3 s
- ☐ Nightly database and storage backups running, copied off-server; one restore tested
- ☐ Local authorisation lists pushed to every site
- ☐ Platform billing account created (`--platform-admin`, operator's own organisation); every customer's plan and private sites set
- ☐ Roaming (if used): `ocpi.` host reachable over https from outside; roaming identity set; first partner's connection test passed; tax treatment of roaming CDRs agreed
- ☐ Fleet billing (if used): seller NPWP/NITKU and e-Faktur items confirmed with the tax adviser; every fleet account has its NPWP and billing e-mail; one test XML imported into Coretax and checked
- ☐ Promotions and memberships (if used): the membership-fee e-Faktur item confirmed; each new promotion tried on a test session; app passes only after the live QRIS acquirer is connected
- ☐ Plug & Charge (if used): the PKI gateway connected, trust anchors fetched and installed, each charger's V2G certificate installed, one test contract charged on each charger model
- ☐ Profile 3 chargers: the charging-station CA on the TLS terminator, Caddy updated from `deploy/Caddyfile` (`X-Client-Cert-Fingerprint` assigned), and one charger connected with its certificate
- ☐ Integrations (if used): the `api.` host answers `/openapi.json` and refuses `/v1/auth/login`; each integrator has a sandbox key for testing and a production key with only the permissions it needs
- ☐ Integrations: the QRIS acquirer connected and its notification URL set in its dashboard, one small real payment confirmed with each enabled method (QRIS, each e-wallet, a card), and, if card holds are on, one hold captured after a real session; if linking is on, one e-wallet linked, paid in one tap and its unused balance refunded; the sign-in code provider (and fallback) connected and a code received on a test phone
- ☐ Driver app: `MAP_TILE_URL` points at your own or a commercial tile service; a test phone received a "charging started" notification; a reservation was honoured by each charger model in use
- ☐ Alert routing: e-mail and WhatsApp test messages received; at least one critical rule to an on-call contact with escalation; a test fault on a bench charger produced the alert, the escalation and the "resolved" message

15. Configuration reference

Variable	Default	Purpose
<code>DATABASE_URL</code>	local dev DB	App connection (runtime role <code>plugsure_app</code>).

MIGRATION_DATABASE_URL	—	Path B only: owner connection for the migrator.
POSTGRES_APP_USER / POSTGRES_APP_PASSWORD	plugsure_app / —	Runtime role provisioned by the migrator.
AUDIT_HMAC_KEY , SECRETS_KEY	— (required)	Audit-log keying; secrets encryption.
INTERNAL_API_TOKEN	—	API ↔ gateway bridge secret.
GATEWAY_INTERNAL_URL	—	API: where the gateway is.
EVENT_RELAY	false	Gateway: relay live events to the API.
RUN_WORKERS	true	Gateway: background workers (incl. webhook delivery, refunds, outage alerts). Set false on extra replicas.
GATEWAY_DIAG	(unset)	Gateway: verbose logs a health summary every 10 s (event-loop delay, database pool, worker passes). Unset, only problems are logged.
OFFLINE_ALERT_MINUTES	15	Gateway: minutes a charger may be offline before a critical alert (and webhook). Resolves itself on reconnect.
CONSOLE_PUBLIC_URL	PUBLIC_BASE_URL	Gateway: console address linked from alert e-mails and WhatsApp messages.
ALERT_TIMEZONE	Asia/Jakarta	Quiet hours and times in alert messages.
BILLING_ISSUER_NAME / BILLING_ISSUER_NPWP	PT. RailSure Solutions Indonesia (PlugSure) / —	API: issuer printed on commission statements.
BILLING_TIMEZONE	Asia/Jakarta	API: where calendar months are cut for statements.
ALERT_NOTIFY_MAX_PER_15MIN	20	Flood guard per recipient and channel; past it, one "messages paused" notice.
OCPI_PUBLIC_URL	PUBLIC_BASE_URL	API and gateway: public origin roaming partners call (versions URL = <this>/ocpi/versions).
OCPI_PBJT_IN_EXCL_VAT	true	API and gateway: count PBJT-TL in a roaming CDR's excl_vat . Confirm with a tax adviser.

OCPI_REALTIME_AUTH_TIMEOUT_MS	5000	Gateway: how long a charger waits for a partner's real-time approval of a card.
MAP_TILE_URL / MAP_ATTRIBUTION	OpenStreetMap tiles / © OpenStreetMap contributors	API: driver-app map tiles ({z}/{x}/{y} , optional {s}) and their credit line. The CSP follows the host.
VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY	empty (generated and stored)	API and gateway: Web Push key pair. Set only to bring an existing key.
VAPID_SUBJECT	mailto:ops@plugsure.id	Gateway: contact given to push services.
PUSH_ALLOWED_HOSTS	FCM, Mozilla, Apple, Windows	Gateway: push services notifications may go to in production (suffix match; SSRF guard).
DRIVER_RESERVATIONS / DRIVER_RESERVATION_MINUTES / DRIVER_RESERVATION_NO_SHOW_LIMIT	true / 15 / 2	API: connector reservations from the app, their length, and unused reservations per 24 h before reserving pauses.
API_HOST / API_PORT / OCPP_PORT	0.0.0.0 / 9200 / 9220	Listen addresses.
API_TRUSTED_PROXIES	empty	Proxies whose X-Forwarded-* headers are believed.
API_RATE_LIMIT_PER_MIN	600	Per-IP request limit.
OCPP_VERSIONS	ocpp1.6	Negotiated protocols.
OCPP_MIN_SECURITY_PROFILE	0	Minimum charger security profile (production: 2).
OCPP_TRUST_PROXY_PROTO	false	Believe Caddy's TLS/cert headers.
OCPP_AUTO_ADOPT	on in development only	Auto-enrol unknown chargers.
OCPP_HEARTBEAT_S	300	Heartbeat interval sent to chargers.
OCPP_KEY_ROTATION_GRACE_MS	86400000	Old key accepted for 24 h after rotation.
OCPP_MAX_MESSAGES_PER_S / OCPP_MESSAGE_BURST	20 / 200	Gateway: per-charger message budget. Past it the charger's messages are paced, never refused or dropped; the burst covers a charger uploading its queue after an outage. 0 disables it.

PUBLIC_BASE_URL , OCPP_PUBLIC_URL	—	URLs handed to chargers / printed on commissioning exports.
STORAGE_DIR	./var/storage	Firmware images and logs (persistent volume).
MAX_FIRMWARE_BYTES / MAX_DIAGNOSTICS_BYTES	512 MB / 200 MB	Upload limits.
FIRMWARE_LINK_GRACE_HOURS	24	API: a charger's firmware download link works while a campaign needs the image, then this many hours after the charger's job last moved.
CONSOLE_COOKIE_SECURE	true in production	HTTPS-only session cookie.
LOGIN_MAX_FAILURES / LOGIN_LOCK_MINUTES / PASSWORD_MIN_LENGTH	5 / 15 / 12	Sign-in policy.
VAULT_ADDR , VAULT_TOKEN , VAULT_PKI_MOUNT , VAULT_PKI_ROLE , VAULT_CERT_TTL	— / pki / ocpp-charger / 8760h	Optional Profile 3 certificate issuance.
CHARGER_CA_CERT_FILE / CHARGER_CA_KEY_FILE	— (created on first use)	Your own CA for charger certificates; otherwise PlugSure creates one (key sealed with SECRETS_KEY).
CHARGER_CERT_DAYS / CHARGER_CERT_RENEW_DAYS	730 / 30	Charger certificate validity; renewal over OCPP this many days before expiry.
CSMS_ROOT_CA_FILE	—	Root of the OCPP host's TLS certificate, handed to chargers as csms-root.pem (empty for a public CA).
PNC_PKI , PNC_PKI_URL , PNC_PKI_TOKEN	none in production	ISO 15118 Plug & Charge: the PKI gateway (deploy/README.md). mock = test PKI, refused in production.
PNC_V2G_SIGNER , PNC_VAULT_MOUNT , PNC_VAULT_ROLE	pki / pki_v2g / secc	Sign chargers' V2G certificates with your own sub-CA in Vault instead of the PKI.
PNC_RENEW_DAYS / PNC_OCSP_TIMEOUT_MS	30 / 5000	V2G certificate renewal lead time; OCSP request timeout.
PPN_RATE_BPS , PPN_DPP_NUM / DEN , PBJT_BASE , PBJT_IN_PPN_BASE	1200, 11/12, energy, true	Tax treatment — confirm with a tax adviser.
PLN_LK_BASE , PLN_CURAH_BASE	1645, 707	PLN base rates for the regulated ceilings.

IDLE_FEE_CAP_IDR	100000	Per-session cap on idle/time fees.
WBP_START / WBP_END	17:00 / 22:00	Peak (WBP) window for time-of-use pricing.

PlugSure CSMS v1.3.0 · Deployment Guide · See also [RELEASE-NOTES-v1.3.0.md](#), [deploy/README.md](#) (full runbook) and [docs/ACCEPTANCE-v1.3.md](#).